

Calcul du temps d'exécution d'un programme

Dispensé par :

Jean Serge Dimitri Ouattara

jean.ouattara@ujkz.bf

Version 2021-2022

Calcul du temps d'exécution d'un programme

- 1 Généralités
- 2 Calcul du temps d'exécution d'un programme
- 3 Comparaison de temps d'exécutions

Généralités

Notions

- Un programme est la traduction d'un algorithme dans un langage exécutable par l'ordinateur ;
- Les langages de programmation classiques doivent être **compilés** ou **interprétés** pour produire un code exécutable ;
- Le code exécutable est fonction du système d'exploitation (Linux Vs Windows) et de l'architecture (par exemple processeur). C'est pourquoi la mesure du temps d'exécution d'un programme n'est déterminante que si l'on précise un certain nombre de choses comme le type de processeur (constructeur, modèle, fréquence), la mémoire (oui je sais c'est pas une contrainte mais ... concrètement si) et le système d'exploitation.

Généralités

Temps d'exécution utilisateur VS Temps d'exécution processeur

Il y a deux mesures de temps d'exécutions à distinguer :

- le **temps d'exécution utilisateur** ou **User Time** correspond à la charge de calcul de l'ordinateur de test. Elle peut **fluctuer** ; un programme fonctionnant seul sur un ordinateur peut avoir un temps d'exécution de 10 min alors que le temps d'exécution peut augmenter de 10% si l'ordinateur est utilisée à naviguer par exemple pendant le déroulement des calculs.
- le **temps d'exécution processeur** ou **CPU Time** est le temps passé par le programme sur le processeur. C'est une constante qui ne dépend pas de la charge de travail de l'ordinateur.

Naturellement il faut travailler avec le CPU Time !

Exemple 1 : Calcul du temps d'exécution d'un programme

```
#include <stdio.h>
#include <time.h>
int main(void){
    float te=0.0;
    clock_t debut, fin;
    debut = clock();
    Ackermann(3,3);
    fin = clock();
    te = (float) (fin-debut)/CLOCKS_PER_SEC;
    printf("Temps d'exécution = %f\n", te);
    return 0;
}
```

Précisions sur l'exemple 1 : Calcul du temps d'exécution d'un programme

- La bibliothèque **time.h** contient **clock_t** et **clock()** ;
- Le nombre de cycles processeur nécessaire à l'exécution est la différence entre les nombres de cycles processeur avant et après l'exécution ;
- Le temps est déterminé par le nombre de cycles processeur par seconde **CLOCKS_PER_SEC** :

$$\text{temps} = \frac{\text{Nombre de cycles processeurs}}{\text{Nombre de cycles processeurs par seconde}}$$

- Pour plus de d'exactitude de calcul utilisez **temps_execution = (float)(t2-t1)/(CLOCKS_PER_SEC/1000000)** ; et appelez plusieurs fois la fonction à tester dans une boucle avant de faire la moyenne des temps d'exécutions.

Exemple 2 : Calcul du temps moyen d'exécution

```
int main(void){
    int i;
    float te=0.0;
    clock_t debut, fin;
    for(i=0;i<10000;i++){
        debut = clock();
        Ackermann(3,3);
        fin = clock();
        temps+= (float) (fin-debut) / (CLOCKS_PER_SEC/1000000);
    }
    te=te/10000;
    printf("Temps d'exécution = %f\n", te);
    return 0;
}
```

Exemple 3 : Calcul de l'évolution du temps d'exécution

```
int main(void) {  
    int i,m,n; float te=0.0; clock_t debut, fin;  
    for(m=0; m<6; m++){  
        for(n=0; n<6; m++){  
            for(i=0; i<10000; i++){  
                debut = clock(); Ackermann(m,n); fin = clock();  
                te+= (float) (fin-debut) / (CLOCKS_PER_SEC/1000000);  
            }  
            te=te/10000;  
            printf("Ack(%d,%d) = %f\n", m,n,te);  
        }  
    }  
    return 0;  
}
```

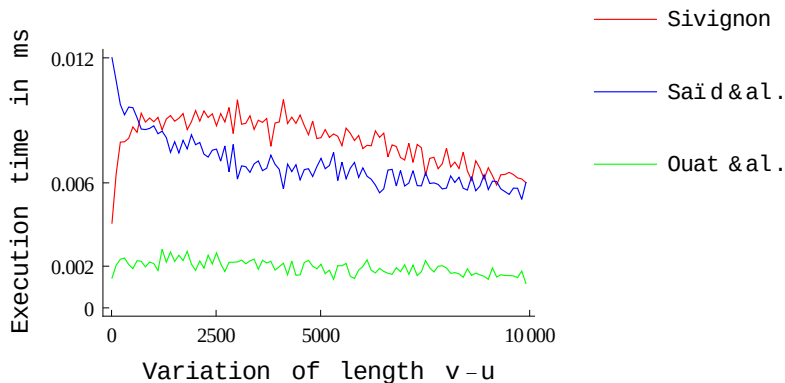

Exemple 4 : comparaison de temps d'exécutions de programmes

```
int main(void){  
    int i; float tel=0.0, te2=0.0;  
    clock_t debut1, debut2, fin1, fin2;  
    for(i=0;i++;i<10000){  
        debut1 = clock(); Ackermann(3,3); fin1 = clock();  
        debut2 = clock(); Fibonaci(3); fin2 = clock();  
        tel+= (float) (fin1-debut1)/(CLOCKS_PER_SEC/1000000);  
        te2+= (float) (fin2-debut2)/(CLOCKS_PER_SEC/1000000);  
    }  
    tel=tel/10000; te2=te2/10000;  
    printf("Ack(3,3) = %f\t Fib(3) = %f\n", tel,te2);  
    return 0;  
}
```

Exemple 5 : Comparaison de l'évolution

```
int main(void){
    int i,n; float te1=0.0, te2=0.0; clock_t debut1, debut2,
    for(n=0; n<4; n++){
        for(i=0; i<10000; i++){
            debut1 = clock(); Ackermann(5,n); fin1 = clock();
            debut2 = clock(); Fibonacci(n); fin2 = clock();
            te1+= (float)(fin1-debut1)/(CLOCKS_PER_SEC/1000000);
            te2+= (float)(fin2-debut2)/(CLOCKS_PER_SEC/1000000);
        }
        te1=te1/10000; te2=te2/10000;
        printf("A(5,%d)=%f\t F(%d)=%f\n", n,n,te1,te2);
    }
    return 0;
}
```

Illustration de l'évolution des temps d'exécution



Exemple 6 : Sortie dans un fichier texte

```
int main(void){
float te;int indice,i,j,a=0;
    clock_t debut, fin, temps;
ft = fopen("resultat.txt", "wt");
    for(i=0;i<10000;i++){
        debut = clock();
        for(indice=0;indice<j;indice++){a++; }
        fin = clock();
        temps+=(fin-debut);
    }
    te = (float)temps/(CLOCKS_PER_SEC*10);
    fprintf(ft, "%.2f\t",te);
    fclose(ft);
}
```

Travail à faire : comparaison du temps d'exécution

Comparaison du temps d'exécution des algorithmes itératif et récursif de calcul de Fibonacci

On souhaite avoir une idée concrète de l'évolution des algorithmes itératif et récursif de calcul de Fibonacci. Pour cela On vous demande d'implémenter :

- 1 Le calcul des temps d'exécution de F_n avec l'implémentation récursive pour n compris entre 1 et 1000 ;
- 2 Le calcul des temps d'exécution de F_n avec l'implémentation itérative pour n compris entre 1 et 1000 ;
- 3 La comparaison des temps d'exécution de F_n dans les implémentations itérative et récursive pour n compris entre 1 et 1000.